Project 2: Market Basket Analysis using Apriori and PCY

Zhuoxin Ge 83511A

March 2, 2026

1 Introduction

Frequent itemset mining is a fundamental problem in data mining. This project implements and compares the Apriori and PCY algorithms for discovering frequent item pairs in a transactional dataset derived from the IMDB Top 1000 Movies dataset.

Each movie is considered a basket, and the actors listed in `Star1`, `Star2`, `Star3`, and `Star4` are treated as items.

The objectives of this project are:

- To correctly implement Apriori and PCY.
- To experimentally compare their efficiency.
- To analyze how performance scales with data size.

2 Dataset Description

The dataset used is the IMDB Top 1000 Movies dataset.

Only the following attributes were considered:

- Star1
- Star2
- Star3
- Star4

After preprocessing:

- Number of baskets: 1000
- Items per basket: 4 actors

Each basket therefore represents a movie and contains four actor names.

3 Data Organization and Preprocessing

The dataset was transformed into transactional form:

- Missing values were removed.
- Whitespace was stripped from actor names.
- Each movie was converted into a set of actors.

This ensures compatibility with frequent itemset mining algorithms.

4 Algorithms

4.1 Apriori

Apriori works in two passes for pair mining:

1. Count frequent 1-itemsets.
2. Generate candidate 2-itemsets from frequent items.
3. Count support and prune infrequent pairs.

Candidate generation follows:

$$C_2 = \{\{i, j\} \mid i, j \in L_1\}$$

4.2 PCY

PCY enhances Apriori by:

- Hashing candidate pairs into buckets during the first pass.
- Using a bitmap to mark frequent buckets.
- Filtering candidate pairs based on bucket frequency before counting.

This reduces memory usage and candidate counting operations.

Apriori suffers from combinatorial explosion as support decreases, while PCY mitigates this through hashing-based pruning.

5 Experimental Setup

The experiments were conducted in Python on Google Colab.

All experiments were executed using the same random seed and identical hardware configuration. The full implementation is provided in the accompanying Jupyter notebook.

Two experimental dimensions were evaluated:

1. Effect of minimum support.
2. Scalability with increasing dataset size.

Number of buckets was set to 5000 unless otherwise stated.

6 Experiment 1: Effect of Minimum Support

Minimum support was varied from 3 to 10.

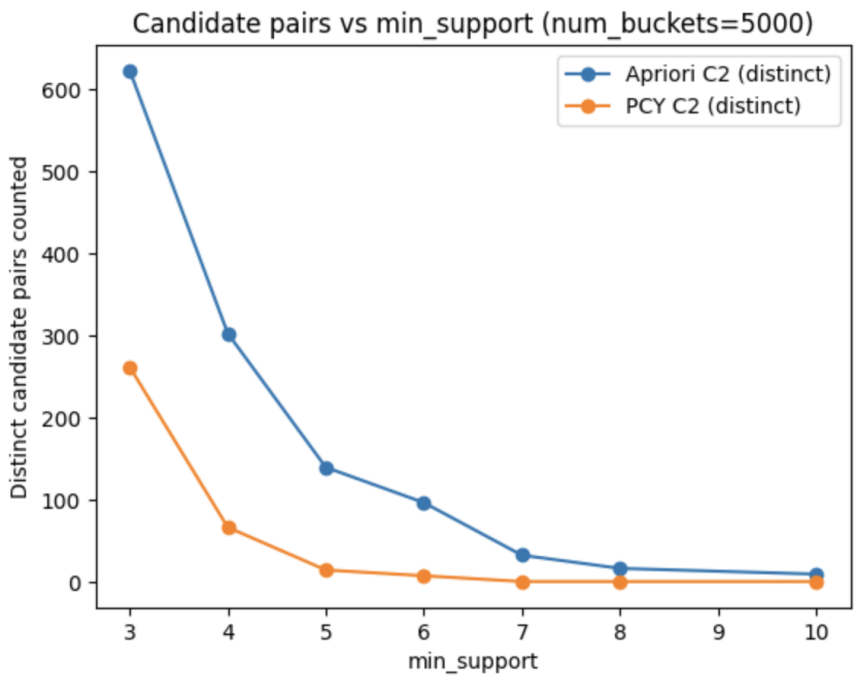

Figure 1: Distinct candidate pairs vs minimum support (5000 buckets)

At support = 3:

- Apriori generated 622 distinct candidate pairs.
- PCY reduced this to 229 pairs.

This corresponds to approximately a 63% reduction in candidate space.

As minimum support increases, both algorithms generate fewer candidates. However, PCY consistently produces significantly fewer candidates due to bucket pruning.

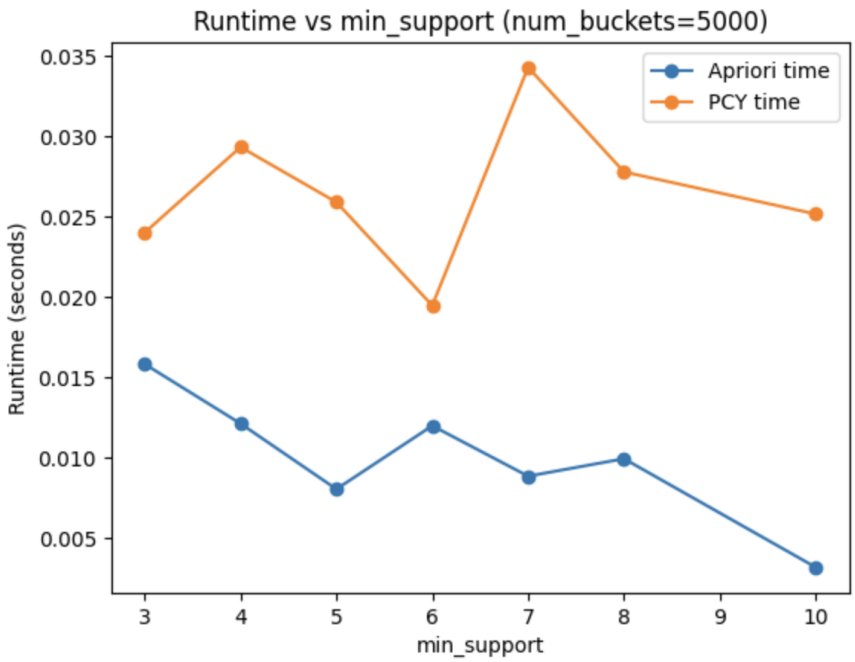

Figure 2: Runtime vs minimum support

Runtime differences are moderate for this dataset size. This is expected since hashing overhead may offset gains when data size is small.

7 Experiment 2: Scalability (Fixed Support and Buckets)

Dataset size was increased by duplicating baskets (2x, 5x, 10x).

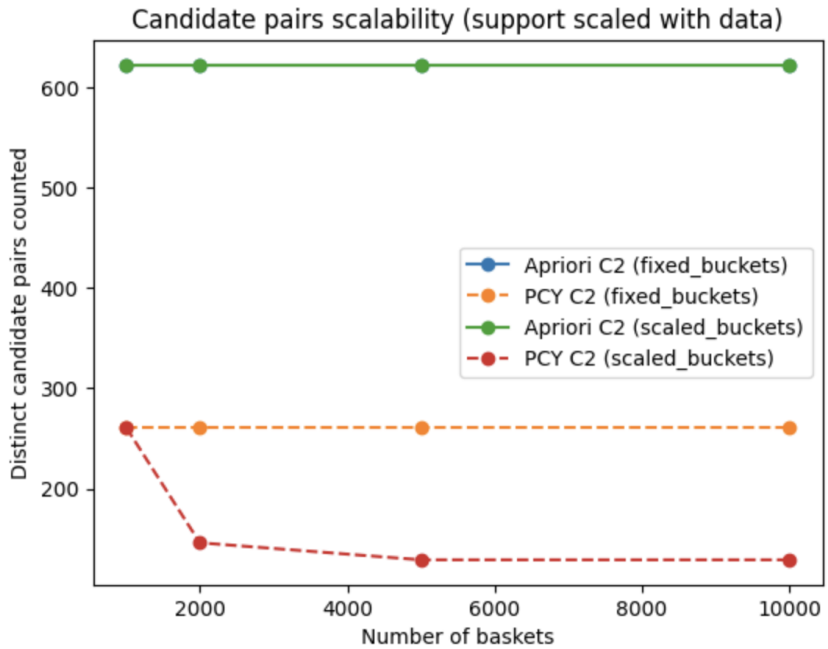

Figure 3: Distinct candidate pairs scalability(fixed support and buckets)

Runtime increases approximately linearly with the number of baskets for both algorithms.

However, when bucket count is fixed, PCY may suffer from increased hash collisions, limiting pruning effectiveness.

8 Experiment 3: Scalability with Scaled Support and Buckets

To better simulate realistic scaling, minimum support and bucket count were scaled proportionally with dataset size.

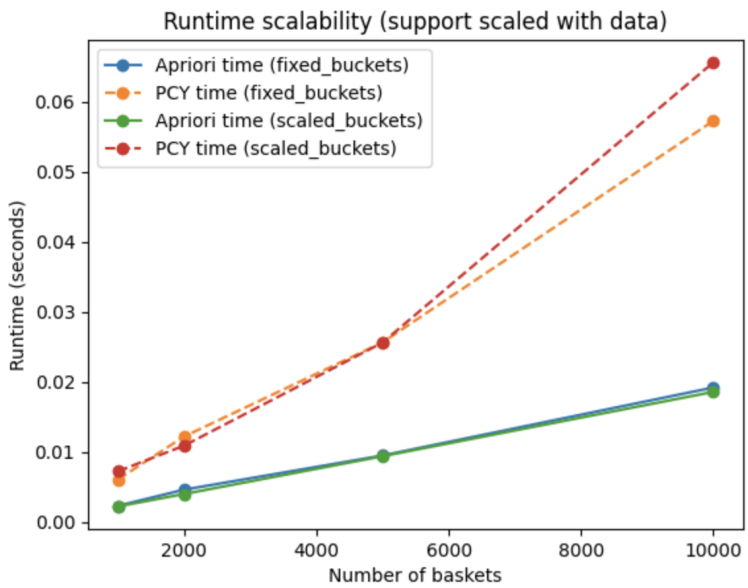

Figure 4: Runtime scalability (scaled support and buckets)

Under scaled configuration:

- Apriori candidate count remains constant (622 pairs).
- PCY candidate count decreases dramatically as bucket count increases.

For example, with 10,000 baskets and 50,000 buckets, PCY reduces candidate pairs to 31.

This confirms the theoretical advantage of PCY: increasing hash space reduces collisions and improves pruning efficiency.

9 Discussion

The experiments confirm theoretical expectations:

- Lower support increases combinatorial candidate generation.
- PCY effectively reduces candidate pairs via hashing.
- For small datasets, runtime improvements are limited.
- For larger datasets with sufficient buckets, PCY demonstrates better scalability.

From a computational perspective, Apriori has time complexity proportional to the number of candidate pairs generated. When the minimum support threshold decreases, the number of frequent 1-itemsets increases, leading to a quadratic growth in candidate 2-itemsets. This explains the rapid increase in candidate counts observed in the experiments.

PCY reduces the effective candidate space by hashing pairs into buckets, thereby pruning many infrequent pairs before explicit counting. When the number of buckets increases proportionally with dataset size, hash collisions decrease, which further improves pruning efficiency.

Both algorithms produce identical frequent itemsets, confirming correctness of implementation.

10 Conclusion

This project implemented and compared Apriori and PCY for frequent pair mining.

Results demonstrate:

- PCY significantly reduces candidate generation.
- Scalability improves when bucket size scales with data.
- Both algorithms return identical frequent itemsets.

PCY becomes increasingly advantageous as dataset size grows, validating its theoretical design. Future work could extend this study to larger real-world transactional datasets.

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.